

ICT ACADEMY OF KERALA

Internship Report

Full Stack Application Development with ReactJS



**Deon M Baby
Goutham Manohar
Mohamed Hafeeb Hassan
Ihjas Ahammed K**

SS College, Areekode

13.2.2025

EXECUTIVE SUMMARY

This internship project focused on developing an **Ecommerce Website**, an online marketplace that allows users to buy and sell products in a secure and user-friendly manner. The project aimed to implement key features such as user authentication, product listing, shopping cart, and an admin panel for management. Utilizing the MERN stack (MongoDB, Express.js, React, and Node.js), we developed a robust and scalable application. The internship helped us gain practical experience in full-stack development, database management, and application security

INTRODUCTION

The e-commerce industry is rapidly growing, with more users opting for online shopping. Our project aimed to build a comprehensive **online marketplace** that provides a seamless user experience for both buyers and sellers. The system ensures secure transactions and an intuitive interface, making online shopping more accessible.

OBJECTIVES

The primary objectives of this project were:

1. To develop an interactive and responsive **Ecommerce Website** using the **MERN stack**.
2. To implement user authentication and authorization.
3. To create a structured product listing system with search and filtering options.
4. To build a secure **shopping cart** and **checkout system** with payment gateway integration.
5. To develop an **admin panel** for managing users and products.
6. To ensure **security, usability, and performance** in the application.

SCOPE AND DELIVERABLES

Scope:

The online marketplace includes the following features:

- **User Registration and Authentication**
- **Product Listings and Search**
- **Shopping Cart and Checkout**
- **User Profiles**
- **Admin Panel for management**

Deliverables:

- A fully functional **Ecommerce Website**.
- Secure login and registration system.
- Product listing functionality with a search feature.
- A working shopping cart and checkout process.
- An admin panel to manage users and products.

METHODOLOGY

The development followed the **Agile methodology**, ensuring iterative development and regular testing. The key steps were:

- **Requirement Analysis:** Understanding user needs and defining system features.
- **Technology Stack Selection:** Using **MERN stack** for development.
- **Database Design:** Creating schemas in **MongoDB** for user and product data.
- **Frontend Development:** Using **ReactJS** for an interactive UI.
- **Backend Development:** Implementing API endpoints with **Node.js** and **Express.js**.
- **Testing & Debugging:** Ensuring functionality and fixing issues.
- **Deployment:** Hosting the application on a cloud platform.

PROJECT ACTIVITIES

- **User Authentication:** Implemented secure **JWT-based authentication**.
- **Product Management:** Developed CRUD functionality for product listings.
- **Shopping Cart & Checkout:** Integrated a cart system and connected a **payment gateway**.
- **User Profiles:** Enabled users to update personal information.
- **Admin Panel:** Provided an interface for managing users and products.

RESULTS & FINDINGS

- Successfully developed and deployed an **Ecommerce Website**.
- Implemented **secure authentication and authorization**.
- Achieved **fast and efficient** product search and browsing.
- Ensured a **responsive and user-friendly interface**.
- Identified and resolved **security vulnerabilities** to prevent unauthorized access.

CONCLUSION

The internship project was a valuable learning experience, enabling us to apply full-stack development concepts in a real-world scenario. We successfully built an Ecommerce Website with key features, ensuring security, usability, and performance. This project enhanced our technical skills in **ReactJS, Node.js, Express.js, and MongoDB** while also improving our problem-solving and teamwork abilities.

APPENDIX

Include any additional supporting documents, samples of your work, project artifacts, or reference materials that are relevant to your internship project.

Data Base Schemas

1. User Schema

```
server > models > JS User.js > ...  
1   const mongoose = require('mongoose');  
2  
3   const userSchema = new mongoose.Schema({  
4     username: { type: String, required: true, unique: true },  
5     password: { type: String, required: true },  
6     email:    { type: String, required: true }  
7   });  
8  
9   module.exports = mongoose.model('User', userSchema);  
10
```

2. Cart Schema

```

server > models > JS Cart.js > ...
1  const mongoose = require('mongoose');
2  const { Schema } = mongoose;
3
4  const ItemSchema = new Schema({
5    id: { type: Number, required: true },
6    image: { type: String, required: true },
7    name: { type: String, required: true },
8    rating: { type: Number, required: true },
9    price: { type: Number, required: true },
10   priceOld: { type: Number }, // optional field
11   description: { type: String, required: true },
12   author: { type: String, required: true }
13 });
14
15 const CartItemSchema = new Schema({
16   item: { type: ItemSchema, required: true },
17   count: { type: Number, required: true, default: 1 }
18 });
19
20 const CartSchema = new Schema({
21   username: { type: String, required: true },
22   cartItems: { type: [CartItemSchema], default: [] }
23 });
24
25 module.exports = mongoose.model('Cart', CartSchema);
26

```

3. Orders Schema

```

server > models > JS Orders.js > ...
1  const mongoose = require('mongoose');
2  const { Schema } = mongoose;
3
4  const ItemSchema = new Schema({
5    id: { type: Number, required: true },
6    image: { type: String, required: true },
7    name: { type: String, required: true },
8    rating: { type: Number, required: true },
9    price: { type: Number, required: true },
10   priceOld: { type: Number },
11   description: { type: String, required: true },
12   author: { type: String, required: true }
13 });
14
15 const OrderSchema = new Schema({
16   item: { type: ItemSchema, required: true },
17   count: { type: Number, required: true },
18   status: { type: String, required: true, default: "Requested" }
19 });
20
21 const Orders = new Schema({
22   username: { type: String, required: true },
23   orders: { type: [OrderSchema], default: [] }
24 });
25
26 module.exports = mongoose.model('Orders', Orders);
27

```

4. Category Schema

```

server > models > JS Category.js > ...
1  const mongoose = require('mongoose');
2
3  const ItemSchema = new mongoose.Schema({
4    id: { type: Number, required: true },
5    image: { type: String, required: true },
6    name: { type: String, required: true },
7    rating: { type: String, required: true },
8    price: { type: String, required: true },
9    priceOld: { type: String, default: null },
10   description: { type: String },
11   author: { type: String }
12 });
13
14 const CategorySchema = new mongoose.Schema({
15   name: { type: String, required: true },
16   items: [ItemSchema]
17 });
18
19 module.exports = mongoose.model('Category', CategorySchema);
20

```

Backend API Endpoints Report

This document provides an overview of the backend API endpoints, their functionality, and expected input/output.

1. User Management

GET /

- **Description:** Root endpoint to check if the server is running.
- **Response:** "Hello"

POST /user

- **Description:** Retrieves a user by username and password.

Input:

```
{
```

```
"username": "exampleUser",  
"password": "examplePass"  
}
```

-
- **Response:** User object or error message.

POST /user/add

- **Description:** Adds a new user.

Input:

```
{  
  
"username": "newUser",  
"password": "newPass",  
"email": "user@example.com"  
}
```

-
- **Response:** Success or error message.

POST /user/setpassword

- **Description:** Updates user password.

Input:

```
{  
  
"username": "user1",  
"email": "user@example.com",  
"oldPassword": "oldPass",  
"newPassword": "newPass"  
}
```

-

- **Response:** Success or error message.

DELETE /user/remove

- **Description:** Deletes a user.

Input:

```
{  
  
  "username": "user1",  
  
  "adminPass": "admin123"  
}
```

-
- **Response:** Success or error message.

POST /users

- **Description:** Retrieves all users (Admin access required).

Input:

```
{  
  
  "adminPass": "admin123"  
}
```

-
- **Response:** List of users or error message.

2. Product Management

POST /product/add

- **Description:** Adds a new product to a category.

Input:

```
{
```

```
"categoryName": "Electronics",  
"product": { "name": "Laptop", "price": 1200 }  
}
```

-
- **Response:** Success message and added product.

PUT /product/update

- **Description:** Updates an existing product.

Input:

```
{  
  
"categoryName": "Electronics",  
  
"productId": "1234",  
  
"updateData": { "price": 1100 }  
}
```

-
- **Response:** Updated product or error message.

DELETE /product/remove

- **Description:** Removes a product from a category.

Input:

```
{  
  
"categoryName": "Electronics",  
  
"productId": "1234"  
}
```

-
- **Response:** Success message.

GET /products

- **Description:** Retrieves all products.
- **Response:** List of all products.

GET /products/:categoryName

- **Description:** Retrieves products from a specific category.
- **Response:** List of products in that category.

GET /product/byname/:productName

- **Description:** Retrieves products by name.
- **Response:** Matching product details.

3. Category Management

POST /category/add

- **Description:** Adds a new category.

Input:

```
{
  "categoryData": { "name": "Electronics" }
}
```

- **Response:** Success message and added category.

PUT /category/update

- **Description:** Updates category information.

Input:

```
{
  "categoryName": "Electronics",
  "updateData": { "description": "New Description" }
}
```

-
- **Response:** Updated category details.

DELETE /category/remove

- **Description:** Deletes a category.

Input:

```
{
  "categoryName": "Electronics"
}
```

- - **Response:** Success message.
-

4. Cart Management

GET /cart/:username

- **Description:** Retrieves the cart of a user.
- **Response:** Cart details.

POST /cart/add

- **Description:** Adds an item to a user's cart.

Input:

```
{
  "username": "user1",
  "item": { "name": "Laptop", "price": 1200 }
}
```

-
- **Response:** Updated cart.

DELETE /cart/remove

- **Description:** Removes an item from the cart.

Input:

```
{  
  
  "username": "user1",  
  
  "itemName": "Laptop"  
}
```

-
- **Response:** Updated cart.

PUT /cart/update

- **Description:** Updates item quantity in the cart.

Input:

```
{  
  
  "username": "user1",  
  
  "itemName": "Laptop",  
  
  "newCount": 2  
}
```

-
- **Response:** Updated cart.

5. Order Management

POST /order/add

- **Description:** Places a new order.

Input:

```
{  
  
  "username": "user1",
```

```
"item": { "name": "Laptop", "price": 1200 },  
"count": 1  
}
```

-
- **Response:** Updated order list.

GET /order/:username

- **Description:** Retrieves all orders of a user.
- **Response:** List of user orders.

DELETE /order/remove

- **Description:** Removes an item from orders.

Input:

```
{  
  
"username": "user1",  
  
"itemName": "Laptop"  
}
```

-
- **Response:** Updated order list.

PUT /order/update

- **Description:** Updates the order status.

Input:

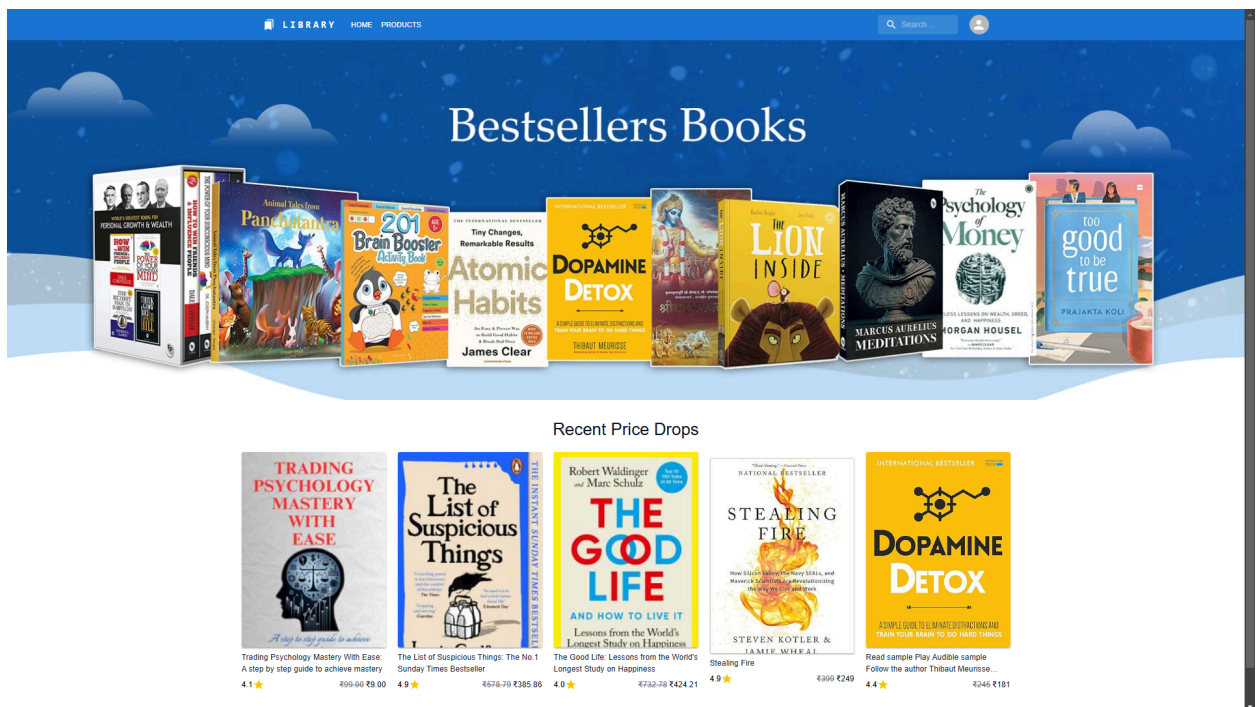
```
{  
  
"username": "user1",  
  
"itemName": "Laptop",  
  
"status": "Shipped"  
}
```

-
- **Response:** Updated order list.

Conclusion

This backend API provides endpoints for user authentication, product management, category management, cart operations, and order processing. Each endpoint has clear input requirements and structured responses to ensure smooth operation.

Wireframes & Mockups:



ihjas
ihjas@gmail.com

admin
admin@google.com

Products Management

ADD CATEGORY

Recent Price Drops

ADD PRODUCT

Trading Psychology M...
Vikash Kumar
\$9.00 \$99.00
Rating: 4.1

The List of Suspicious...
Jennie Godfrey
\$385.98 \$578.79
Rating: 4.9

The Good Life: Lesso...
Robert Waldinger and Marc Schulz
\$424.21 \$732.78
Rating: 4.0

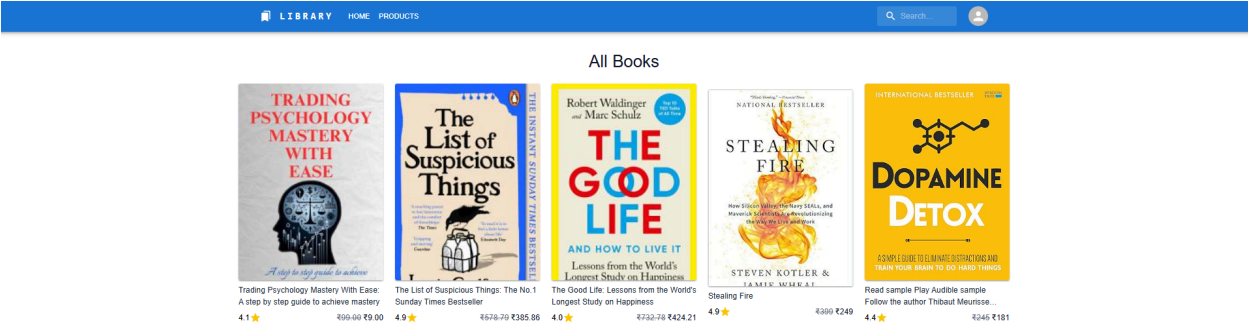
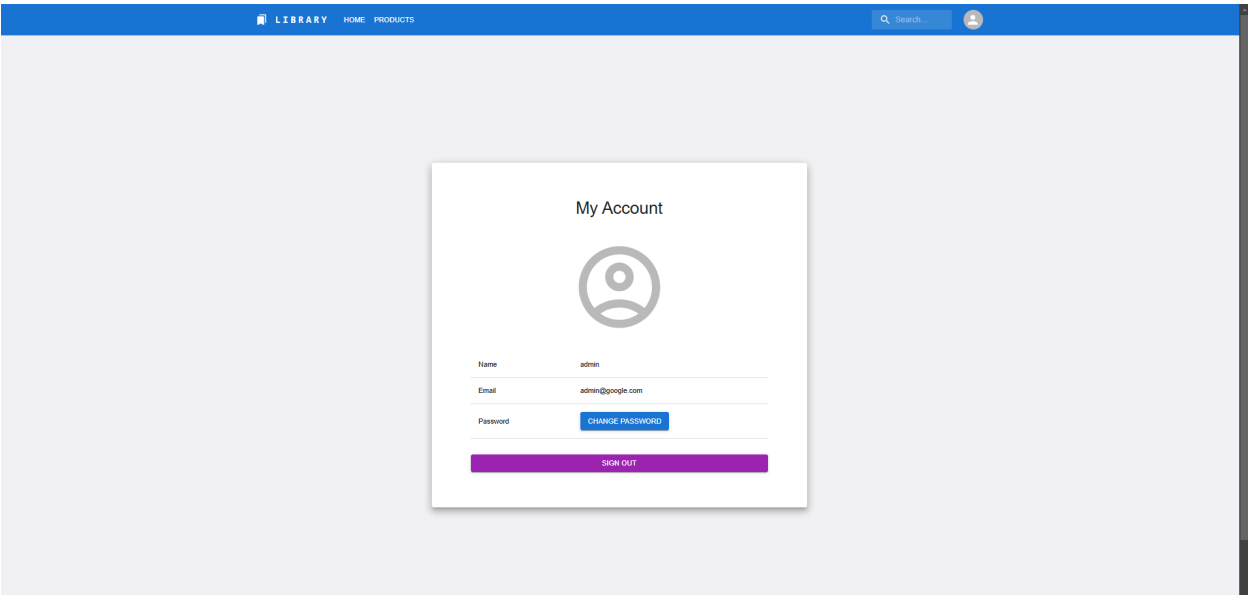
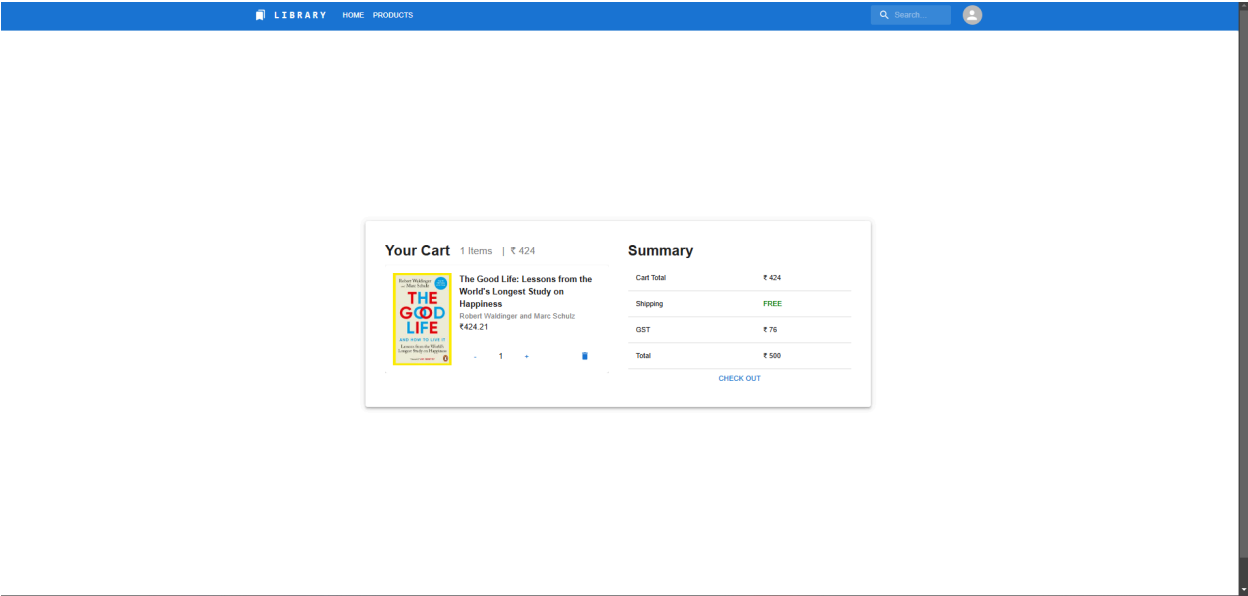
Stealing Fire
Steven Kotler
\$249 \$399
Rating: 4.9

Read sample Play Au...
Thibaut Meunier
\$181 \$245
Rating: 4.4

My Orders

The Good Life: Lessons from the
World's Longest Study on Happiness
Robert Waldinger and Marc Schulz
\$424.21

Status
Requested
[CLOSE ORDER](#)



LIBRARYHOMEPRODUCTS

Search...fire

Search Results for 'fire'

STEALING FIRE

How Steven Kotler, Gary Shtetler, and
Hanna & Nicholas Layton are working
to save the world and their
lives

STEVEN KOTLER &
JAMIE MCKAY

Stealing Fire

4.9★

€399 €249

Copyright © 2025

LIBRARYHOMEPRODUCTS

Search...

SIGN UP

Username

Email

Password

SIGN UP

Already have an account? [Sign In](#)

Copyright © 2025

LIBRARYHOMEPRODUCTS

Search...

LOGIN

Username

Password

SIGN IN

Don't have an account? [Sign Up](#)

Copyright © 2025

1. Version Control:

The project was maintained using Git, and the repository can be accessed at:

<https://github.com/ihjas-ahammed/book-store>